

INTE2547-2580 Security Testing

Assignment 3

	Assessment type: Individual assignment. Submit online via Canvas→Assignments→Assignment 3. Marks awarded for meeting requirements as closely as possible. Clarifications/updates may be made via announcements/relevant discussion forums.
	Due date: Week 12, Friday 23:59PM Assignments received late and without prior extension approval or special consideration will be penalised by a deduction of 10% of the total score possible per calendar day late for that assessment.
	Weighting: 100 points (Contributes 30% of total mark)

1. Overview

The objective of Assignment 3 is to evaluate your knowledge on the topics covered by Lectures 1 - 12. Topics include SQL inject attacks, Cross-site Scripting attacks, and AI Security. **You are required to prepare the solutions (recording if required and reports with screenshots + explanations).** Then upload them as a single ZIP to CANVAS.

2. Course learning outcomes

This assessment is relevant to the following course learning outcomes:

- CLO2 Investigate and model the possible vulnerabilities and threats for a given application system
- CLO3 Design, implement test procedures and perform post-testing evaluation
- CLO4 Analyse and evaluate security related scenarios

3. Submission

You must follow the following special instructions:

- You must use the values provided in the questions.
- Hand-written answers are not allowed and will not be assessed. Compose your answers using any word processing software (e.g. MS Word).
- You are required to show all the steps and intermediate results for each question.
- Upload your solution as a **single ZIP file (PDF/Word Doc + demo video)** in CANVAS.
- This assessment will determine your ability to:
- Follow requirements provided in this document and in the lessons.
- Independently solve a problem by using security concepts taught over the first four weeks of the course.

- Meeting deadlines.

After the due date, you will have 5 business days to submit your assignment as a late submission. Assignments received late and without prior extension approval or special consideration will be penalised by a deduction of 10% of the total score possible per calendar day late for that assessment.

4. Assessment details

Please ensure that you have read **Sections 1 to 3** of this document before going further. Assessment details are provided in the next page.

Section 1: Exploiting Web Services using SQL Injection Attacks (50 marks)

SQL injection is a code injection technique that exploits the vulnerabilities in the interface between web applications and database servers. In this section, we have created a web application given in the **Labsetup-sql.zip** Docker, which is vulnerable to the SQL injection attack <http://www.seed-server.com/>.

Our web application includes the common mistakes made by many web developers. Students' goal is to find ways to exploit the SQL injection vulnerabilities, demonstrate the damage that can be achieved by the attack.

The login credential to MySQL database is: Username: root Password: dees.

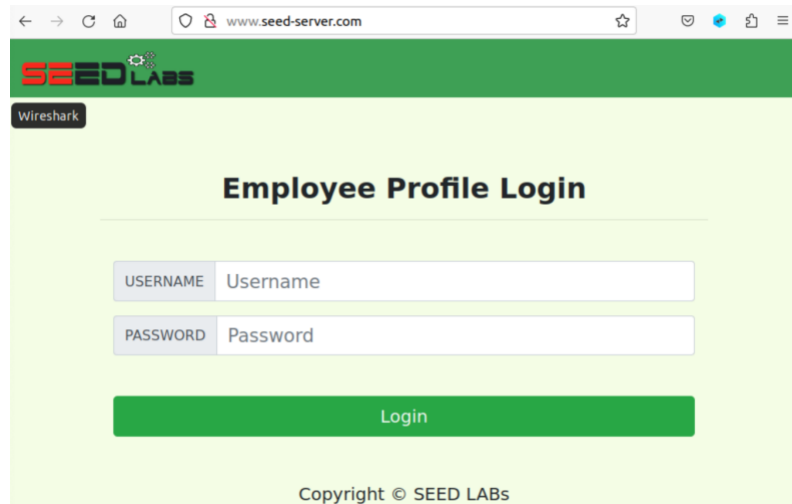


Figure 1 Login page to the web application

Assume you are an attacker who attempts to exploit the web application without knowing any user's password. Please conduct the following SQL injection attacks in the following sub-questions.

Write a report for the following sub-questions with **screenshots** and **explanations**, as well as a **video recording** to demonstrate.

(Note: In this task, you are going to use a salary SALARY_1 and a password PWD_1 defined by your student No. Suppose your student No. is 's3123456', then the SALARY_1 is '3456' and PWD_1 is 's3123456'.)

You need to manage to log into <http://www.seed-server.com/> without providing a password. The authentication method is implemented in `unsafe_home.php` located in `/var/www/SQL_Injection`. The following code snippet shows how users are authenticated.

```
$input_uname = $_GET['username'];
$input_pwd = $_GET['Password'];
$hashed_pwd = sha1($input_pwd);
...
$sql = "SELECT id, name, eid, salary, birth, ssn, address, email,
        nickname, Password
        FROM credential
        WHERE name= '$input_uname' and Password='$hashed_pwd'";
$result = $conn -> query($sql);
```

Q1.1 (10 marks) In the above SQL statement, the variable `$input_uname` holds the string typed in

the Username textbox, and \$hashed_pwd holds the string typed in the Password textbox. User's inputs in these two textboxes are placed directly in the SQL query string. There is a SQL-injection vulnerability in the above SELECT query. Exploit the vulnerability and log into Ted's account without knowing the correct password. Demonstrate that your attack succeeds and explain your attack based on your exploited SQL.

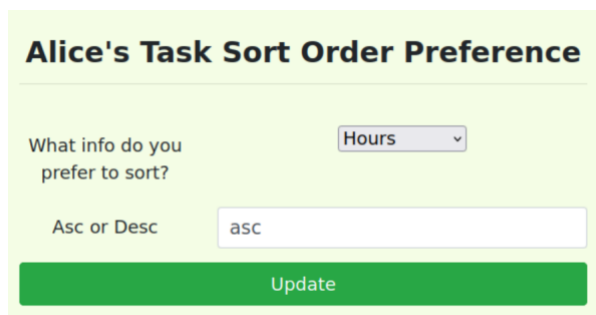
You also need to make an unauthorised modification to the database. Your goal is to modify another user's profile using SQL injections. In this web application, if users want to update their profiles, they can click the Edit Profile link on the navigation bar, and then fill out a form to update the profile information. After the user sends the update request to the server, an UPDATE SQL statement will be constructed in unsafe_edit_backend.php in the /image_www/Code folder.

```
$hashed_pwd = sha1($input_pwd);  
$sql = "UPDATE credential SET  
    nickname='$input_nickname',  
    email='$input_email',  
    address='$input_address',  
    Password='$hashed_pwd',  
    PhoneNumber='$input_phonenumber'  
WHERE ID=$id;";  
$conn->query($sql);
```

Q1.2 (10 marks) In above sub-question Q1.1, you have exploited the web application and logged in as Ted. Exploiting the vulnerability in this UPDATE SQL statement by using Ted's account to modify Bobby's salary to SALARY_1 without knowing Ted's password and Bobby's password. Demonstrate that your attack succeeds and explain your attack based on your exploited SQL.

Q1.3 (10 marks) In above sub-question Q1.1, you have exploited the web application and logged in as Ted. Exploiting the vulnerability in this UPDATE SQL statement by using Ted's account to modify Bobby's nickname to your student name, email to your RMIT student email, and address to "RMIT", without knowing Ted's password and Bobby's password. Demonstrate that your attack succeeds and explain your attack based on your exploited SQL.

In a normal scenario, an user can update his/her view preference with sorting by asc or desc in the Set View Preference page http://www.seed-server.com/unsafe_view_order.php as shown in Figure 2. Afterwards, the View Tasks page http://www.seed-server.com/unsafe_tasks_view.php will show the user's Declared Tasks as shown in Figure 3.



Alice's Task Sort Order Preference

What info do you prefer to sort?

Asc or Desc

Figure 2 Task Sort Order Preference page of the web application

Alice's Declared Tasks					
Task Name	Task Owner	Hours	Amount	Task Description	Type
On-site client requirement collection	Alice	50	10000	Collect client requirement	Collaboration
Business Analysis	Alice	50	4000	Task Breakdown	Individual
Demo App to client	Alice	50	2000	Onsite demonstration	Individual
Design System Architecture1	Alice	30	5000	Design components and communication protocols	Collaboration

Figure 3 user's Declared Tasks page of the web application

However, the web application is vulnerable to SQL injection attack where attacker turns one SQL statement into two SQL statements, with the second one being a malicious SQL statement. You need to perform the attack to display all the tasks of the user who has the maximum number of tasks when you view your tasks.

Note that you **may** use the database tables

`tasks (TaskID, Name, Hours, Amount, Description, Owner, Type) ,`
`preference (PreferenceID, favourite, Owner) ,` and `credential (ID) .`

MySQL functions `userIdMaxTasks ()` returns the ID of an user who has the maximum number of tasks in your database.

Q1.4 (10 marks) In the above two web pages, you need to identify the vulnerable SQL queries that are exploitable to allow the attacker to conduct the above attack: “display all the tasks of the user who has the maximum number of tasks when you view your tasks”. Show the identifies vulnerabilities and explain why they are exploitable to allow for the attack.

Q1.5 (10 marks) You need to exploit the identifies vulnerable SQL queries in above Q1.5 to conduct the attack: “display all the tasks of the user who has the maximum number of tasks when you view your tasks”. Demonstrate that your attack succeeds and explain your attack based on your exploited SQL.

Section 2: Exploiting Web Services using Cross-Site Scripting Attacks (40 marks)

Cross-site scripting (XSS) is a type of vulnerability commonly found in web applications. This vulnerability makes it possible for attackers to inject malicious code (e.g. JavaScript programs) into victim's web browser. In this section, we have created a web application `Elgg` given in the **Labsetup-xss.zip** Docker, which is vulnerable to the XSS attack <http://www.seed-server.com/>. `Elgg` is a very popular open-source web application for social network. In this section, students need to exploit this vulnerability to launch an XSS attack on the `Elgg`.

We have created several user accounts on the `Elgg` server; the user name and passwords are given in the following.

UserName	Password
admin	seedadmin
alice	seedalice
boby	seedboby
charlie	seedcharlie
samy	seedsamy

Assume you are an attacker who attempts to exploit the web. Please conduct the following XSS attacks in the following sub-questions.

Write a report for the following sub-questions with screenshots and explanations, as well as a video recording to demonstrate.

Q2.1 (10 marks) You need to embed a JavaScript program in the “About me” field of `Samy`’s profile, such that when another user (e.g., `Alice`) views `Samy`’s profile, the JavaScript program will be executed and an alert window will be displayed with message “Attack with XSS by ”+your student no.. For example, if your student no. is s3123456, then the message is “Attack with XSS by s3123456”. Demonstrate that your attack succeeds and explain your attack based on your injected scripts.

Q2.2 (10 marks) You need to embed a JavaScript program in the “About me” field of `Samy`’s profile, such that when another user (e.g., `Alice`) views `Samy`’s profile, the user’s cookies will be displayed in the alert window. Demonstrate that your attack succeeds and explain your attack based on your injected scripts.

Q2.3 (10 marks) You need to modify the victim’s (e.g., `Alice`) profile when the victim visits `Samy`’s page. We will write an XSS worm to complete the task.

We need to write a malicious JavaScript program that forges HTTP requests directly from the victim’s browser, without the intervention of the attacker. To modify profile, we should first find out how a legitimate user edits or modifies his/her profile in `Elgg`. More specifically, we need to figure out how the HTTP POST request is constructed to modify a user’s profile. Afterwards, we can write a JavaScript program to send out the same HTTP request. We provide a skeleton JavaScript code that aids in completing the task.

There are three “FILL-IN” fields. The description field should be your RMIT student no. For the rests, you need to figure out the values to those fields by using Firefox’s HTTP inspection tool to view the HTTP POST request when modifying `Samy`’s profile. Demonstrate your answer and how you find out the answer.

```

<script type="text/javascript">
window.onload=function(){
    //JavaScript code to access user name, user guid, Time Stamp
    __elgg_ts
    //and Security Token __elgg_token
    var userName=elgg.session.user.name;
    var guid="&guid="+elgg.session.user.guid;
    var ts="__elgg_ts="+elgg.security.token.__elgg_ts;
    var token="__elgg_token="+elgg.security.token.__elgg_token;
    var desc="&description= *** "; //FILL-IN
    var name="&name="+userName;
    //Construct the content of your url.
    var sendurl="***"; //FILL-IN
    var content=token+ts+name+desc+guid;
    var samyGuid=***; //FILL-IN
    if(elgg.session.user.guid!=samyGuid) {
        //Create and send Ajax request to modify profile
        var Ajax=null;
        Ajax=new XMLHttpRequest();
        Ajax.open("POST",sendurl,true)
        Ajax.setRequestHeader("Host","www.seed-server.com");
        Ajax.setRequestHeader("Content-Type","application/x-www-
form-urlencoded");
        Ajax.send(content);
    }
}
</script>

```

Q2.4 (10 marks) In the above sub-question Q2.3, you already construct the XSS worm. Afterwards, you need to embed a JavaScript program in the “About me” field of Samy’s profile, such that when another user (e.g., Alice) views Samy’s profile, the above script will be executed and the description (i.e., your student no.) will be injected into Alice’s profile automatically. Demonstrate that your attack succeeds and explain your attack based on your screenshots.

Section 3: Analyzing the security of AI systems. (10 marks)

AI applications have emerged recently and provided various services to facilitate our daily life. Existing studies show that current AI applications are vulnerable to various attacks, and they are suffering a series of security issues.

Image classification is a typical application of current AI models. Please use the MNIST model provided on <https://kennysong.github.io/adversarial.js/> as the target model, and choose the “Jacobian-based Saliency map attack (stronger)” as the attack method to generate the adversarial samples for evasion attacks. The original image should be the image with “3” and the parameter of “Turn this image into a” is 8.

Q3.1 (5 marks) Provide screenshots to report your observations.

Q3.2 (5 marks) Analyze the reasons (e.g., image similarity) for this successful evasion attack (hints: refer to the attack patterns in Page 19 of Week 6 lecture).

5. Academic integrity and plagiarism

Academic integrity is about the honest presentation of your academic work. It means acknowledging the work of others while developing your own insights, knowledge and ideas.

You should take extreme care that you have:

- Acknowledged words, data, diagrams, models, frameworks and/or ideas of others you have quoted (i.e., directly copied), summarised, paraphrased, discussed or mentioned in your assessment through the appropriate referencing methods
- Provided a reference list of the publication details so your reader can locate the source if necessary. This includes material taken from Internet sites

If you do not acknowledge the sources of your material, you may be accused of plagiarism because you have presented the work and ideas of another person without appropriate referencing, as if they were your own.

RMIT University treats plagiarism as a very serious offence constituting misconduct.

Plagiarism covers a variety of inappropriate behaviours, including:

- Failure to properly document a source
- Copyright material from the Internet or databases
- Collusion between students

For further information on our policies and procedures, please refer to the University website.

6. Assessment declaration

When you submit work electronically, you agree to the [assessment declaration](#).

Criteria	Rating						Total
Section 1 SQL Injection Attacks	Any 5 of questions Q1.1-Q1.5 are answered correctly.	Any 4 of questions Q1.1-Q1.5 are answered correctly.	Any 3 of questions Q1.1-Q1.5 are answered correctly.	Any 2 of questions Q1.1-Q1.5 are answered correctly.	Any 1 of questions Q1.1-Q1.5 are answered correctly.	Answers are completely incorrect	
	SQL Injection attacks are correctly performed in a clear way with video, screenshots, and description in the 5 questions.	SQL Injection attacks are correctly performed in a clear way with video, screenshots, and description in the 4 questions.	SQL Injection attacks are correctly performed in a clear way with video, screenshots, and description in the 3 questions.	SQL Injection attacks are correctly performed in a clear way with video, screenshots, and description in the 2 questions.	SQL Injection attacks are correctly performed in a clear way with video, screenshots, and description in the 1 question.	Or	
	Vulnerabilities and attack strategies are correctly explained in a clear way, with screenshots in the 5 questions.	Vulnerabilities and attack strategies are correctly explained in a clear way, with screenshots in the 4 questions.	Vulnerabilities and attack strategies are correctly explained in a clear way, with screenshots in the 3 questions.	Vulnerabilities and attack strategies are correctly explained in a clear way, with screenshots in the 2 questions.	Vulnerabilities and attack strategies are correctly explained in a clear way, with screenshots in the 1 question.	Not answered	
	Justification is reasonable, clear, and accurately presented in the 5 questions.	Justification is reasonable, clear, and accurately presented in the 4 questions.	Justification is reasonable, clear, and accurately presented in the 3 questions.	Justification is reasonable, clear, and accurately presented in the 2 questions.	Justification is reasonable, clear, and accurately presented in the 1 question.		
	50 marks	40 marks	30 marks	20 marks	10 marks	0 mark	50 marks
Section 2 XSS Attacks	Questions Q2.1-Q2.4 are answered correctly.	Any 3 of questions Q2.1-Q2.4 are answered correctly.	Any 2 of questions Q2.1-Q2.4 are answered correctly.	Any 1 of questions Q2.1-Q2.4 are answered correctly.	Answers are completely incorrect		
	XSS attacks are correctly performed in a clear way with video, screenshots, and description.	XSS attacks are correctly performed in a clear way with video, screenshots, and description in the 3 questions.	XSS attacks are correctly performed in a clear way with video, screenshots, and description in the 2 questions.	XSS attacks are correctly performed in a clear way with video, screenshots, and description in the 1 question.	Or		
	Vulnerabilities and attack strategies are correctly explained in a clear way, with screenshots.	Vulnerabilities and attack strategies are correctly explained in a clear way, with screenshots in the 3 questions.	Vulnerabilities and attack strategies are correctly explained in a clear way, with screenshots in the 2 questions.	Vulnerabilities and attack strategies are correctly explained in a clear way, with screenshots in the 1 question.	Not answered		
	Justification is reasonable, clear, and accurately presented.	Justification is reasonable, clear, and accurately presented in the 3 questions.	Justification is reasonable, clear, and accurately presented in the 2 questions.	Justification is reasonable, clear, and accurately presented in the 1 question.			
	40 marks	30 marks	20 marks	10 marks	0 mark	40 marks	
Section 3 AI Security	Questions Q3.1-Q3.2 are answered correctly.	Questions Q3.1-Q3.2 are answered correctly.	Questions Q3.1-Q3.2 are answered correctly.	Questions Q3.1-Q3.2 are answered correctly.	Questions Q3.1-Q3.2 are answered correctly.	Answers are completely incorrect	

	Adversarial attacks are correctly performed in a clear way with video, screenshots, and description. Vulnerabilities are correctly explained in a clear way. Justification is reasonable, clear, and accurately presented.	Adversarial attacks are correctly performed in a clear way with video, screenshots, and description. Vulnerabilities are correctly explained in a clear way. Justification is not reasonable, clear, and accurately presented.	Adversarial attacks are correctly performed in a clear way with video, screenshots, and description. Vulnerabilities are not correctly explained in a clear way. Justification is not reasonable, clear, and accurately presented.	Adversarial attacks are correctly performed in a clear way with video, screenshots. The description is not provided. Vulnerabilities are not correctly explained in a clear way. Justification is reasonable, clear, and accurately presented.	Or Not answered	
	10 marks	8 marks	5 marks	3 marks	0 mark	10 marks